

The Sub-4-Bit Frontier: BitNet & QuIP#, Worked Out by Hand

TERNARY WEIGHTS, 1.58 BITS, AND A MATMUL WITH (ALMOST) NO MULTIPLIES

What this document does. It works the ternary frontier by hand. First it shows why a $\{-1, 0, +1\}$ weight costs exactly $\log_2 3 = 1.58$ bits and how five of them pack into one byte. Then it computes a real dot product two ways — standard FP16 (8 multiplies) versus the BitNet *multiply-free* accumulation (1 multiply, the rest adds and sign-flips) — and measures the gap. Finally it lays out the memory and accuracy frontier across INT3, INT2, ternary, and QuIP#. Every figure is reproduced in Appendix A (`m02_t06_sub4bit_bitnet_engine.py`).

The one idea. Below 4 bits, uniform per-group scales stop working — the rounding error outgrows what error-compensation can fix. Two escape routes exist: *non-uniform vector codebooks* (QuIP#, AQLM) that quantize whole weight *vectors* to lattice points, and *training from scratch* at low precision (BitNet), where the network learns representations that already assume ternary weights. The ternary case is special: a weight in $\{-1, 0, +1\}$ turns every multiply into an add, a subtract, or a skip.

CONCEPTS COVERED, IN ORDER

the information content of a ternary weight ($\log_2 3 = 1.58$ bits) and bit-packing; BitNet ternarization with an absmean scale; the multiply-free ternary dot product vs. FP16; the memory frontier (INT3/INT2/ternary); why sub-INT4 needs codebooks (QuIP#) or from-scratch training; the accuracy frontier and the BitNet hardware caveat.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The object under study	2
2	Step 1 — Why ternary costs 1.58 bits	2
3	Step 2 — Ternarize the weights	2
4	Step 3 — The multiply-free dot product	3
5	Step 4 — The memory frontier	4
6	Step 5 — QuIP# and the accuracy frontier	4
7	The argument, at a glance	5
7.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The object under study

A single neuron’s dot product: one ternary weight vector against one activation vector. Small enough to do every operation by hand, structured exactly like a row of a BitNet linear layer.

Quantity	Symbol	Value here
Weights (FP16)	W	$[0.45, -0.62, 0.08, 0.51, -0.40, 0.05, -0.55, 0.33]$
Activations	x	$[1.20, -0.30, 0.90, 0.40, 1.10, -0.70, 0.60, 0.20]$
Ternary alphabet	—	$\{-1, 0, +1\}$
BitNet scale	γ	$\text{mean} W $

Intuition

A full-precision weight tells you “multiply by 0.45.” A ternary weight only tells you a *direction*: add this input, subtract it, or ignore it. You throw away the magnitude of each individual weight and recover it globally with one shared scale γ . It sounds reckless — and for post-training it is — but a network *trained* to live with ternary weights learns to encode magnitude in *how many* weights point each way, not in any single one.

2 Step 1 — Why ternary costs 1.58 bits

A binary weight has 2 states = 1 bit. A ternary weight has 3 states. Information content is the log of the number of states:

Bits per ternary weight

$$\log_2 3 = 1.585 \text{ bits.}$$

Packing ternary weights into bytes

You cannot store 1.58 bits directly, but you can pack several ternary symbols together. Five ternary weights have $3^5 = 243$ joint states, and $243 \leq 256 = 2^8$, so five weights fit in one byte:

$$\frac{8 \text{ bits}}{5 \text{ weights}} = 1.60 \text{ bits/weight,}$$

essentially the 1.58 floor. This is why BitNet b1.58 is named for 1.58 bits, not 2. ✓

3 Step 2 — Ternarize the weights

BitNet uses a single *absmean* scale and rounds each weight to the nearest ternary value:

BitNet ternarization

$$\gamma = \frac{1}{n} \sum_i |W_i|, \quad W_i^{\text{tern}} = \text{clip}\left(\text{round}\left(\frac{W_i}{\gamma}\right), -1, +1\right).$$

Apply it to our vector

$$\gamma = \text{mean}|W| = 0.3738.$$

Dividing each weight by γ and rounding to $\{-1, 0, +1\}$:

$$W^{\text{tern}} = [+1, -1, 0, +1, -1, 0, -1, +1].$$

The two small weights (0.08, 0.05) collapse to 0 (skip); the rest keep only their sign. ✓

4 Step 3 — The multiply-free dot product

Here is the payoff. The FP16 dot product needs a real multiply per element. The ternary one needs *one* multiply for the whole vector.

Ternary matmul — one scale, the rest adds

$$y = W \cdot x \approx \gamma \sum_i W_i^{\text{tern}} x_i = \gamma \left(\sum_{W_i^{\text{tern}}=+1} x_i - \sum_{W_i^{\text{tern}}=-1} x_i \right).$$

Accumulate by hand

add x_i where $w=+1$ ($i = 0, 3, 7$) : $1.20 + 0.40 + 0.20 = +1.80$,
 subtract x_i where $w=-1$ ($i = 1, 4, 6$) : $0.30 + 1.10 + 0.60 = 1.40$,
 skip x_i where $w=0$ ($i = 2, 5$) : (free, 2 terms),
 accumulator = $1.80 - 1.40 = 0.40$,
 $y_{\text{tern}} = \gamma \times 0.40 = 0.3738 \times 0.40 = 0.1495$.

Total real multiplies: 1 (the final γ), versus 8 for FP16. The inner loop is pure integer add/subtract/skip. ✓

Multiply-free is the whole promise

On hardware that can exploit it, replacing n multiply-accumulates with n adds and one scale is a large energy and area win — multipliers are far more expensive than adders in silicon. A ternary matmul is, at heart, “count how many inputs point up minus how many point down, then scale once.”

Watch out

Compare the two answers: FP16 $y = 0.2630$ versus ternary $y = 0.1495$ — a 43% error. That is *not* a bug; it is the honest cost of throwing away per-weight magnitude by *post-hoc* ternarization. It is exactly why you cannot take a trained FP16 model and ternarize it: the error compounds across layers into catastrophe. BitNet works only because it is **trained from scratch** with ternary weights in the forward pass, so the network learns to be accurate *given* the constraint — encoding magnitude in the population of signs, not in individuals.

5 Step 4 — The memory frontier

Each bit shaved off compounds into the 70B footprint:

Format	bits/weight	70B size	compression vs. FP16
FP16	16	140 GB	1.0×
INT4	4	35.0 GB	4.0×
INT3	3	26.2 GB	5.3×
INT2	2	17.5 GB	8.0×
ternary 1.58	1.58	13.9 GB	10.1×

Ternary squeezes a 70B model into under 14 GB — it fits on a single consumer GPU, and in the memory-bound decode regime that 10× fewer bytes is up to 10× less HBM traffic per token.

Watch out

Group-wise INT3/INT2 (a per-group scale every 64–128 weights) is the *post-training* path to sub-4-bit, but uniform scales fray fast: at INT2 the rounding error grows faster than GPTQ/AWQ compensation can absorb. This is why the genuinely good sub-INT4 results come from *non-uniform* methods, not finer uniform grids.

6 Step 5 — QuIP# and the accuracy frontier

QuIP# quantizes weight *vectors* to entries of a carefully designed lattice *codebook*, not individual scalars. Two ideas make it work: (1) trained-LLM weights lie on a low-dimensional manifold, so a vector codebook captures the distribution far better than scalar rounding; (2) pre-multiplying by a random orthogonal (Hadamard) matrix makes the weights *incoherent* — more Gaussian — which is exactly where lattice quantization excels.

Method	bits/wt	Llama-3-70B MMLU	Δ from FP16
FP16 baseline	16	79.5	—
AWQ	4	79.2	−0.3
QuIP#	3	78.7	−0.8 (3-bit sweet spot)
QuIP#	2	76.9	−2.6 (best 2-bit)
AQLM	2	74.8	−4.7
BitNet b1.58	1.58	76.3	−3.2 (from scratch)

Where the frontier actually is

4-bit (AWQ/GPTQ) is essentially free (−0.3 to −0.5). 3-bit QuIP# is the practical edge of post-training quantization (−0.8). At 2-bit and below, even the best methods pay 2–5 points, and ternary requires training from scratch. The takeaway: **3-bit is the current sweet spot** for squeezing further without retraining; ternary is a different game that pays off only with custom-trained models and hardware that exploits multiply-free matmuls.

Watch out

BitNet’s efficiency depends on kernels that skip multiplies. On today’s NVIDIA GPUs the speedup is real but modest ($\sim 2\times$ over FP16) because tensor cores are not built for ternary arithmetic; the full promise needs custom silicon or repurposed accumulators. For 2026 production, BitNet is realistic only if you can train from scratch — you cannot PTQ an FP16 model to 1.58 bits, as Step 3’s 43% dot-product error already foreshadows.

7 The argument, at a glance

#	Step	Result
1	Information	ternary = $\log_2 3 = 1.58$ bits; 5/byte = 1.60 b/w
2	Ternarize	$\gamma=0.374$; $W^{\text{tern}}=[+1, -1, 0, +1, -1, 0, -1, +1]$
3	Multiply-free	1 multiply vs 8; $y_{\text{tern}}=0.150$ vs FP16 0.263
4	Memory	ternary $\rightarrow$ 13.9 GB (10.1 $\times$) for 70B
5	Frontier	3-bit QuIP# -0.8; 2-bit -2.6; BitNet from-scratch

7.1 Equation cheat-sheet

Ternary bits	$\log_2 3 = 1.585$
Packing	$3^5 = 243 \leq 2^8 \Rightarrow 5$ weights/byte
BitNet scale	$\gamma = \frac{1}{n} \sum_i W_i $
Ternarize	$W_i^{\text{tern}} = \text{clip}(\text{round}(W_i/\gamma), -1, 1)$
Multiply-free matmul	$y = \gamma(\sum_{+1} x_i - \sum_{-1} x_i)$
70B at b bits	$70 \times 10^9 \cdot b/8$ bytes

Appendix A — Reference implementation

Every number above is produced by `m02_t06_sub4bit_bitnet_engine.py`. Run it to reproduce the bit count, the ternarization, the multiply-free dot product, and the memory/accuracy frontier; change W or x to explore.

```

1  """
2  Reference implementation for: Module 02 - Topic 6
3  "The sub-4-bit frontier: ternary BitNet (1.58-bit), INT3/INT2, QuIP#."
4
5  We show (1) why ternary weights cost  $\log_2(3) = 1.58$  bits, (2) a multiply-FREE
6  ternary dot product (only adds, subtracts, and a single scale) versus the FP16
7  dot product, and (3) the information / memory math for sub-4-bit formats.
8
9  Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
10 """
11
12 import numpy as np
13
14 np.set_printoptions(precision=4, suppress=True)
15
16 # -----
17 # 1. The information content of a ternary weight.
18 # -----
19 bits_ternary = np.log2(3)
20 print("== 1. Why ternary = 1.58 bits ==")
21 print(f"a ternary symbol {{-1,0,+1}} has 3 states ->  $\log_2(3) = \{{\text{bits\_ternary:.4f}}\}$  bits")
22 print(f"packing: 5 ternary weights =  $3^5 = \{{3**5}\}$  states  $\leq 2^8 = 256$ ,")
23 print(f"so 5 weights fit in 1 byte ->  $\{{8/5:.2f}\}$  bits/weight in practice")
24
25 # -----
26 # 2. BitNet b1.58 quantization: ternarize weights with a single scale.
27 #   scale = mean(|W|); W_tern = round(W/scale) clamped to {-1,0,+1}.
28 # -----
29 W = np.array([0.45, -0.62, 0.08, 0.51, -0.40, 0.05, -0.55, 0.33])
30 x = np.array([1.20, -0.30, 0.90, 0.40, 1.10, -0.70, 0.60, 0.20]) # an activation vector
31 print("\n== 2. BitNet b1.58 ternarization ==")

```

```

32 print("W =", W)
33 print("x =", x)
34 scale = np.mean(np.abs(W)) # BitNet absmean scale
35 W_tern = np.clip(np.round(W / scale), -1, 1)
36 print(f"scale gamma = mean(|W|) = {scale:.4f}")
37 print("W_tern =", W_tern.astype(int), " (each weight is -1, 0, or +1)")
38
39 # -----
40 # 3. The dot product: y = W . x in FP16 vs ternary multiply-free.
41 # -----
42 y_fp16 = float(W @ x)
43 print("\n== 3. Dot product: FP16 vs ternary multiply-free ==")
44 print(f"FP16: y = sum(W_i * x_i) = {y_fp16:.4f} ({W.size} real multiplies + adds)")
45
46 # Ternary: y = scale * sum( sign_i * x_i ), where sign_i in {-1,0,+1}.
47 # No multiplies on the activations -- just add x_i where w=+1, subtract where w=-1, skip where 0.
48 pos = x[W_tern == 1].sum()
49 neg = x[W_tern == -1].sum()
50 skipped = int((W_tern == 0).sum())
51 acc = pos - neg
52 y_tern = scale * acc
53 print("ternary multiply-free accumulation:")
54 print(f" add x_i where w=+1: indices {np.where(W_tern==1)[0].tolist()} -> +{pos:.4f}")
55 print(f" sub x_i where w=-1: indices {np.where(W_tern==-1)[0].tolist()} -> -{neg:.4f}")
56 print(f" skip x_i where w= 0: {skipped} terms (free)")
57 print(f" accumulator = {pos:.4f} - {neg:.4f} = {acc:.4f}")
58 print(f" y = gamma * acc = {scale:.4f} * {acc:.4f} = {y_tern:.4f}")
59 print(f" TOTAL real multiplies = 1 (the final scale), vs {W.size} in FP16")
60 print(f"\n FP16 y = {y_fp16:.4f}")
61 print(f" tern y = {y_tern:.4f}")
62 print(f" abs error = {abs(y_fp16 - y_tern):.4f} rel error = {abs(y_fp16-y_tern)/abs(y_fp16)*100:.1f}%")
63
64 # -----
65 # 4. Memory: ternary vs other widths for a 70B model.
66 # -----
67 print("\n== 4. Memory footprint of a 70B model ==")
68 for name, bpw in [("FP16", 16), ("INT4", 4), ("INT3", 3), ("INT2", 2),
69                  ("ternary 1.58", bits_ternary)]:
70     gb = 70e9 * bpw / 8 / 1e9
71     print(f" {name:14s}: {bpw:5.2f} bits/weight -> {gb:6.1f} GB ({16/bpw:.1f}x vs FP16)")
72
73 # -----
74 # 5. Accuracy frontier (reported, Llama-3-70B class) -- context for the cost.
75 # -----
76 print("\n== 5. The sub-4-bit accuracy frontier (reported MMLU) ==")
77 rows = [
78     ("FP16 baseline", 16, 79.5, 0.0, "-"),
79     ("AWQ", 4, 79.2, -0.3, "production default"),
80     ("QuIP#", 3, 78.7, -0.8, "3-bit sweet spot"),
81     ("QuIP#", 2, 76.9, -2.6, "best 2-bit (lattice codebook)"),
82     ("AQLM", 2, 74.8, -4.7, "codebook, slow calibration"),
83     ("BitNet b1.58", bits_ternary, 76.3, -3.2, "trained from scratch"),
84 ]
85 for name, bpw, mmlu, delta, note in rows:
86     print(f" {name:14s} {bpw:5.2f}b MMLU {mmlu:5.1f} ({delta:+.1f}) {note}")
87 print("\n Note: BitNet must be TRAINED ternary from scratch; you cannot PTQ an")
88 print(" FP16 model to 1.58 bits without catastrophic loss.")

```